

A NUMA-aware Type Extension with Introspective Typing

ANONYMOUS AUTHOR(S)

SUBMISSION ID: 461*

Abstract

Heterogeneous memory architectures are becoming increasingly common in modern computing systems. However, traditional programming models fall short in providing fine-grained control over data placement, instead relying on automated mechanisms that often make suboptimal decisions. In particular, in a Non-Uniform Memory Access (NUMA) system, memory access latency and bandwidth can vary dramatically across the memory subsystem, and incorrect placements can impose significant performance penalties when data and compute are not co-located. We propose to give programmers control over the locality of their data using an introspective typing system for C++ that embeds memory locality directly into the type system. Our approach introduces a `numa` type for data placement, enabling explicit and composable expression of memory and thread affinity at the language level. To support our technique, we extend the `clang++` compiler with a source-to-source transformation that generates specialized versions of user-defined types, recursively retyping member fields to enforce NUMA-aware memory allocation.

We provide a formal description of our extended type system and a proof for *memory locality* in Coq. Moreover, we validate our design with a transactional binary search tree benchmark and a NUMA-aware YCSB benchmark, demonstrating that our system improves performance by up to 31% by aligning memory and computation locality with minimal programmer effort.

ACM Reference Format:

Anonymous Author(s). 2026. A NUMA-aware Type Extension with Introspective Typing. 1, 1 (March 2026), 25 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

As computing systems evolve, the traditional memory model, where memory is treated as a uniform storage space, is becoming increasingly obsolete. The emergence of heterogeneous memory architectures, particularly Non-Uniform Memory Access (NUMA) systems, requires programmers to care about the literal physical location of their data. Unlike conventional architectures, NUMA systems utilize multiple memory devices divided into multiple nodes, each managed by a distinct CPU, and each with its own access latency, bandwidth characteristics, and locality constraints. Depending on how the memory devices and compute nodes are attached to each other, latency and bandwidth can vary dramatically, with significant performance differences between local DRAM accesses managed by the local CPU to remote accesses managed by other sockets. While NUMA-aware memory allocation has been well studied [10, 17, 23, 26], an intuitive programming model that controls fine-grained data placement via language-level abstractions remains an open challenge.

Efficient utilization of NUMA memory is particularly critical in large-scale multi-socket servers, high-performance computing (HPC), and real-time systems where memory locality and access latency directly impact application performance. Traditional programming models generally assume

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/3-ART

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

a flat, uniform address space, leading to unpredictable performance degradation when data locality is not carefully managed. Without explicit programmer control over memory allocation and access patterns, applications running on NUMA architectures frequently suffer from cross-node memory access overhead and unbalanced memory bandwidth utilization. The difference between local and remote accesses can differ by up to 3x [30]. Moreover, emerging Compute Express Link (CXL) devices, which enable memory pooling [37] and disaggregated computing [25], are expected to introduce additional complexity in managing data placement as remote memory moves increasingly further away from the local core — such novel architectures envision data sharing across racks, with the associated performance penalty on access latency.

Existing NUMA-aware solutions are typically ad-hoc and application-specific, and require administrators to tune their applications manually through OS-level memory policies [1, 3]. However, these approaches operate at a rough granularity (at the administrator/application level) and fail to preserve the original programmer’s intent or knowledge. While it is likely that the programmer has expectations about the co-locality of various data objects and structures, this intuition is lost and unleveraged using simple OS policies. Alternatively, the programmer can rely on low-level OS API calls (e.g. libnuma [4]) or simply rely on default OS behavior, but such mechanisms are non-portable, require substantial code refactoring, and are difficult to compose with existing libraries.

As a solution to this problem, we propose an *introspective type system* that embeds memory placement policies directly in the type system of the C++ programming language, implemented as compiler-supported parameterized types. This approach enables fine-grained control over memory locality while enabling NUMA-aware programming at the application level. Our numa type system allows the programmer to declare, at variable declaration, to which NUMA socket (or virtual NUMA socket) the object belongs to. Our system propagates this information across the type system using reflection (ensuring, for instance, that object fields are also appropriately tagged numa). To create data managed by the system, programmers use a numa wrapper, `numa<T, N>`. For instance, to declare a NUMA-resident queue type on (virtual) NUMA node 1, they declare it of type `numa<queue, 1>`. If then, they declare a thread of type `thread_numa<N>`, in this case `thread_numa<1>`, the node ID `N` serves as an indication that this thread and the queue it is about to work on belong together in the same NUMA node.

The use of the introspective type system means the compiler and runtime can be aware of programmer-intended placement policies, both dynamically and statically. Statically, we use the node ID information to do locality aware type propagation on the member fields of the type (e.g., the `numa<queue>`’s head pointer). Our clang-based implementation of the introspective type system is a light-weight source-to-source transform analogous to template expansion. As the transform is implemented in the front-end only and its output is pure C++, it requires no changes to the actual compiler, and most NUMA-aware optimizations can be expressed in explicit C++ code within the transformed output.

While we do not believe we have implemented all possible optimizations, simple NUMA-aware measures can create substantial performance benefits by, for instance, ensuring NUMA objects use a NUMA-pinned allocator and all threads manage a separate per-node cache of free blocks. Using such well-studied optimizations is trivial once the system stack is aware of the underlying data placement intention of the programmer. We also recognize our introspective system could be applied to other memory placement specifications (e.g. to annotate data types as secret to request additional security protection via secure memory or persistent for placement in nonvolatile memory) and expect to follow these lines in future work.

Our contributions include:

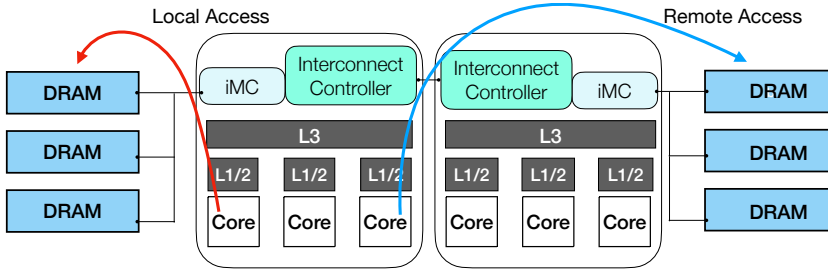


Fig. 1. Generic NUMA Architecture.

- A novel introspective type system for associating physical memory type with a data object, which propagates through its member fields and dynamic memory allocation behavior.
- An implementation of our introspective types using the clang front-end for C++, along with an associated NUMA backend for fast allocation.
- A lightweight formal model and an operational semantics of NUMA-aware C++, and the proof of *memory locality* demonstrating that a given numa tagged object will be entirely allocated on a particular node.
- An extensive empirical evaluation demonstrating that the numa type system can improve program performance with little effort in common scenarios, namely, in large transactional workloads and key-value stores.

Our paper proceeds as follows. Section 2 provides necessary background on NUMA-aware programming and particularities of C++ important to our implementation. Section 3 introduces our introspective type system at the API level, while Section 4 provides implementation details of our clang-based tooling. Section 5 provides a proof of correctness for our system when applied to a subset of the C++ language. Section 6 provides experimental results. We compare against related work in Section 7, and conclude with Section 8.

2 Background

2.1 Non-Uniform Memory Access

Non-Uniform Memory Access (NUMA) is an emergent behavior that arises in multiprocessor systems, where the time it takes for a single processor to access memory depends on the memory's physical location relative to the processor. This variation in latency comes from the hardware layout of the NUMA system (see Figure 1). All processors are attached to their own physical memory device, managed by their own integrated memory controller (iMC), but can also access other memory devices connected to other processors. Due to this physical layout, accesses to a memory device attached to the originating CPU (a *local access*) will be substantially faster than those on a different CPU (a *remote access*). The difference between local and remote accesses can be up to 3x for both latency and bandwidth [31]). The *non-uniform* in NUMA describes the disparity in this memory access time across the CPU/memory units (called *nodes*).

NUMA architectures emerged as a solution to the scalability challenges of modern multicore systems; by connecting multiple physical memory devices to operate as a unified system, we can significantly increase memory capacity than what could have been achieved with a single memory chip alone. Further, as systems grow to include more processors, a centralized memory model becomes a performance bottleneck as multiple processors competing for the same memory resource leads to increased contention and reduced efficiency, but multiple devices solves this problem.

However NUMA implies that from a processor's perspective, not all addresses are equal. It is better for a processor to keep its frequently accessed data in local memory as much as possible so it does not have to incur the latency and bandwidth penalty of accessing remote memory. To assist with this problem, operating systems abstract the NUMA hardware into a software model to allow programs access to the underlying topology.

2.1.1 The First Touch Policy. In Linux, the OS abstracts each hardware NUMA node into a software node [2]. Associated with each software NUMA node, it keeps a free-page list, in-use page list and an ordered zone list (DMA, NORMAL, MOVABLE) for each software node. By default, Linux will try to allocate memory from the appropriate zone list of a node from which a CPU request came from, that is, the OS defaults to local allocation (termed a *first-touch policy*). If the request is not satisfied by the zone on local node, the kernel will examine zone lists of other nodes and assigns the allocation to the first node that has a zone list that satisfies the requested address, resulting in remote allocation. When following this policy, and assuming no task migration, the operating system will tend to keep an application's memory local. But task migration for load balancing purposes is inevitable and may eventually result in the separation of a process from its memory.

2.1.2 AutoNUMA. Additionally, Linux has a feature called *AutoNUMA* [18], which can migrate both tasks and pages between NUMA nodes. To keep a task and the memory it uses on the same node, a kernel thread will keep track of the node where a page resides and also tracks the task(s) accessing it using periodic page faults. When it is time for a given task to be scheduled, the kernel thread will provide per-task statistics to the scheduler. If a task's accesses to most of its pages are remote, it will be migrated over to the node where the remote pages reside. Alternatively, a task may have scattered pages across multiple nodes, leaving it no obvious node for scheduling. To alleviate this issue, pages must be collected from each node and brought closer to the task accessing them as well. For this, another kernel thread is responsible for copying pages waiting to be migrated to a target node. AutoNUMA can be useful at the process granularity, but the page based-granularity of the approach, in the absence of additional coordination, means that application objects, even if they have a particular affinity for a NUMA node, will be mixed together on the same page, resulting in poor page placement. We explore this issue in our results (Section 6).

2.2 Templates in C++

We recognize that C++ templates are a widely understood paradigm. However, because our design heavily leverages their specific properties, we outline them in detail in this section to ground the subsequent discussion of our system. A template allows a function or class to operate with different data types providing a powerful mechanism for writing generic and reusable code. When templated code is compiled, the compiler substitutes in the types (and in some cases, values) passed as parameters to a template. This process, called *template expansion*, results in several copies of the original template code, each varying according to the passed parameters.

For example, the C++ standard library contains the `vector<T>` type, which consists of a resizable array of type `T`. If the user creates variables of types `vector<int>` and `vector<char>`, through template expansion, the code for `vector` will be compiled twice, one copy defining the placeholder string `T` as `int` and once using `char`. This mechanism is very useful — type specialization occurs at compile time, not runtime, at the cost of increased code size. C++ also supports *template specialization*, which allows developers to define custom behavior for specific template parameters (e.g. the `vector<bool>` type is a specialized implementation that uses a tightly-packed bitmap), instead of the default template behavior.

3 Design

The goal of our introspective type system is a mechanism of capturing the programmer's intent that is minimally obtrusive yet (1) enables awareness of NUMA locality requirements within the compiler and runtime stack and (2) prevents the misuse of such tags when impossible to enforce. Of course, the proper tool for expressing the semantic meaning of data is the type.

For our work, we target the C++ language as it is a common systems programming language with a large amount of legacy code. C++ is also an interesting target in that it is a language with an irregular and loose type system that requires changes to be implemented carefully.

More specifically, we argue that the principle programming abstraction for data stored in heterogeneous memory should be a *compiler-supported, parameterized, and polymorphic type*. The parameterized type “boxes” an existing type inside additional semantics, namely, those it derives from its storage within a particular memory technology. As inspiration, we use the C++ atomic type-qualifier, which allows a programmer to designate a variable as subject to concurrent accesses.

Given this inspiration, we propose a new type qualifier, `numa`.

3.1 NUMA API and Requirements

Data. To express memory allocation preference, a programmer can use the `numa<T,N>` type when declaring a variable, where `T` is any primitive or user-defined data type, and `N` is the node ID that specifies which NUMA node `T` is to be allocated on. `numa` is a template-like class that gets the type and node ID specified by the user. A variable of type `numa<T,N>` can be thought of as being similar to the type `T` except that the object allocation is strictly constrained to node `N`.

Threads. To express thread placement, we introduce the `thread_numa<N>` type, where `N` is the NUMA node ID to which the thread is to be pinned. `thread_numa` is a template class that instantiates an `std::thread` and internally performs thread pinning to the specified NUMA node. The `thread_numa` class is a simple wrapper, requiring no language changes, but its existence is useful for reasoning at compile time about local and remote accesses.

```

1 #include <numatype.h>
2 ...
3 struct Point{double x,y;};
4 struct Line{Point p1,p2;};
5 struct Triangle{Line a,b,c;};
6 ...
7 numa<Triangle, 0>* t0 = new numa<Triangle, 0>;
8 numa<Triangle, 1>* t1 = new numa<Triangle, 1>;
9
10 check_is_numa(t0->a.p1.x, 0); // statically known allocation location
11 numa_memcpy(&t0->a,&t0->b); // local copy
12 numa_memcpy(&t1->a,&t0->a); // remote copy (use prefetch optimization?)
13
14 //t0 = *t1 // ERR-prohibit casting across numa nodes
15 //numa<Triangle, 2> t2; // ERR-no stack allocation
16 Triangle* t = t0; // allowed, but lost optimization opportunity

```

Fig. 2. Simple numa problems

Usage. Figure 2 provides a simple set of types that illustrate a variety of minimal requirements for our `numa` types, contained in a single `numatype.h` header.

In the figure, a `Triangle` object contains `Line` objects, which in turn each contain `Point` objects consisting of a pair of doubles. The programmer then creates a `Triangle` on NUMA node 0 using `numa<Triangle,0>`. Subsequent manipulations include checking where an object is stored and possibly a NUMA-aware `memcpy`.

From this minimal example, we can begin to extract requirements on the `numa` type extension. First, `numa` objects cannot reside on the stack, as the stack could otherwise become discontinuous — this is an important safety property. Secondly, `memcpy`, and other cross node accesses are likely possibilities for optimization, but require some method of static dispatch, particularly in the C++ case where type information on primitives is lost. Implicit casting from a `numa<T>` object to the `T` should be allowed, but may result in lost optimization opportunities. Finally, all fields within a `numa` object are also `numa` allocated (as are their fields, recursively) and we would like to get this information statically — this is a difficult requirement to enforce as it requires *type introspection* at compile time, that is, it requires the type system to reason about the actual *contents* of types.

While implementing a new type-qualifier is simpler in more modern languages, in C++ this feature is difficult. C++ does indeed support a rich template metaprogramming mechanism which, while powerful for compile-time computation and type manipulation, cannot be used to recursively introspect and transform the member variable types of arbitrary user-defined classes. This limitation arises because C++ lacks native compile-time reflection capabilities, that is, it does not provide a way to enumerate or access the names and types of a class's data members without explicit user intervention. Libraries like `Boost.Hana` [20] or `Boost.Fusion` [33] attempt to address this limitation, but they require additional annotations or metadata in the class definitions, which undermines our full automation ideal. Similarly, despite the fact that C++'s standard template library allows for the templatization of memory allocators on many of its own types, such an approach is not applicable automatically to user defined types.

Given these restrictions, we were led to a true language extension using a front-end clang-based approach described in this section.

3.2 Overall System

Our `numa` type qualifier is a new keyword within the C++ language which cannot be expressed as a template, despite (intentional) appearances to the contrary. The `numa` type qualifier expresses a programmer intention with regards to memory locality, and should be respected throughout the system stack.

Our implementation of this type qualifier uses a clang-based front-end to expand the C++ language. The essence of the solution is a source-to-source conversion which mirrors C++'s native template expansion. The input is a C++ program that uses the `numa` qualifier as though the qualifier could be replaced by its boxed type. The output is a program in purely valid C++ with definitions for all `numa` type specializations, with additional changes to allow for appropriate casting between types and optimizations in `numa` specialized code.

As introduced in the above example, such an implementation must deal with several issues, namely (1) expansion — the definition of a new qualified type `numa<T>` based on the original class `T`, (2) introspective expansion — the recursive application of the `numa` qualifier to all member fields of a class when redefined, (3) the correct handling of casting between the new `numa<T>` type and the original class `T` (4) a safety guarantee that `numa` objects are only declared where they can be appropriately allocated (e.g. not on the stack), and (5) appropriate handling of pointers within a `numa` object, particularly those that may point outside the object itself (and thus to non-`numa` pinned memory). These steps mirror our compilation pipeline for the type, so we present them in this order.

3.3 Simple Template-based Expansion

The first step of our numa type compilation is the expansion of any numa type declaration into a full class declaration. This expansion follows standard practice for C++ template expansion, a source-to-source transform where, the unspecialized, generic, template code, parameterized on some value T , is replicated and T is replaced now with a known value. That is, a C++ declaration of `vector<int> v;` will result in a full replication of the parameterized `vector<T>` class in source, with T replaced with `int`.

After this first expansion, almost every unique `numa<T,N>` declaration in the original code will result in a new class declaration for the appropriate type. We call this expansion the numa declaration of class T . If T is a primitive or pointer type, we simply use a template expansion for this step as the complications of introspection, casting, and dispatch (next subsections) are not needed.

Trivially, any numa object should be allocated in some numa aware way, so handles to the expanded class's new and delete series operators are added for the appropriate interposition.

3.4 Introspective Expansion

The numa type-qualifier indicates that, for a particular object, its allocation is restricted to a particular NUMA node, implying that its member variables are also allocated on said node (and should be appropriately typed). Put another way, any numa class declaration's fields should also have a numa qualifier. The application of the numa qualifier to a field may add a new declaration, which may again trigger a numa specialization. As previously mentioned, C++ disallows introspection within the template system, so we manually execute this task using a pass in clang.

The result of these two passes is shown in Figure 3. Note the numa declaration of a class T can also be understood to be a template specialization of the numa template to class T , and as such, is *almost* valid C++ code assuming an empty `numa<T,N>` definition. All that remains are a few corner cases that result from casting between the qualified and unqualified types, which we cover next.

3.5 Casting

An additional design goal of our system was to allow the casting of numa objects back into the original type, as, for instance, a `numa<Triangle,0>` object has an *is-a* relationship with the original `Triangle` class. However, after the above expansion, we have broken this relationship — the `numa<Triangle,0>` type is unrelated to the unqualified `Triangle`! We fix this problem by adding type coercions in our compiler pass.

Fortunately, by design, both qualified and unqualified types have the same field layout in memory, and further, fields in C++ are accessed using offsets from the start of the object. As a consequence, regardless of a type, a given member in either object shares the same offset and name. As such, it is safe to reinterpret_cast, that is, coerce the type, between two pointers of the qualified and

```

17 // empty numa template specialized
    for each type
18 template<typename T, int N>
19 struct numa{};
20
21 // added by our pass
22 // through expansions
23 struct numa<Point,0>{
24     numa<double,0> x, y;
25     // overload new and delete
26 };
27 struct numa<Line,0>{
28     numa<Point,0> p1, p2;
29     // overload new and delete
30 };
31 struct numa<Triangle,0>{
32     numa<Line,0> a, b, c;
33     // overload new and delete
34 };
35 ...
36 // triggering declaration
37 numa<Triangle, 0>* t;

```

Fig. 3. A numa introspective expansion, triggered by the declaration of `t`. The empty numa template is declared in our header to allow the numa declaration of `t`.

unqualified type. Our tool adds these casts from the numa type to the matched original type as needed. Figure 4 shows this problem and the implemented solution.

```

38 // original code, where user added numa Point
39 numa<Point, 0>* p0;
40 p0->x = 42.0;
41 Point* p = p0; // cast unsupported naively
42 cout<< p0->x;
43
44 // after our pass
45 numa<Point, 0>* p0; // triggers numa expansion
46 p0->x = 42.0;
47 Point* p = reinterpret_cast<Point*>(p0); // cast added by our tool
48 cout<< p0->x; // still prints 42.0

```

Fig. 4. Type casting taking advantage of shared layouts

As C++ already contains a mechanism for the “is-a” relationship, the reader may be wondering why we did not simply use an inheritance relationship between the original type (the parent, e.g. Point) and the NUMA qualified type (the child, e.g. numa<Point, 0>). The problem is that a child object cannot redeclare a parent field as a different type. Consequently, the child cannot change the type of the parent, resulting in either the child having non- numa members (preventing optimizations), or replicating the parent’s fields in its own declaration, which, even worse causes problems if the object is subsequently cast to the parent type — accesses using the parent type will access the non- numa parent fields.

3.6 Method Dispatch

Casting also introduces another problem with respect to method dispatch for member functions. In C++, function dispatch of object methods is generally based on the statically declared type. However, as seen above, we would like the numa object to be castable into the original type, as it could dramatically reduce required code changes. Of course, if the methods are the same between the NUMA qualified and unqualified types, dispatch is not an issue, but the goal of our numa qualifier is to introduce optimizations on the qualified type, thus necessitating a differentiation.

Given these requirements, we would like to dispatch *at runtime* the appropriate method for a given unqualified, original, reference. That is, given, for instance, a Stack*, we would like to call the correct push() method, even if the pointer actually points to a numa<Stack, 0> — this behavior is, unfortunately, not the C++ default.

That said, C++ does have a (non-default) feature, called *virtual* functions, which are used for the runtime dispatch of functions involved in inheritance. Unlike normal functions, which are dispatched according to declared type, a virtual function is dispatched at runtime by the *vtable*. The vtable is a small set of function pointers attached to the end of the object, effectively functioning as member variables. If a virtual method is called on an object, it is looked up using an offset into the object’s vtable, then called. By aligning the vtables of NUMA qualified and unqualified types, we can ensure the numa -aware functions are called even if used on an unqualified reference.

This mechanism does require all methods (except the constructor) in the original type to be marked virtual.

3.7 Safety

After all numa declarations have been placed and subsequently inferred, and all casts have been fixed, the output of our source-to-source transformation is written in pure C++. Our transformation process enforces two safety invariants with help of the C++ type system:

- (1) A numa object is never allocated on the stack: as thread stacks are assumed to be contiguous, a numa stack-allocated object is nonsensical and unsupportable. We catch any attempts by the the programmer to allocate numa objects on the stack during the template expansion time and throw an error.
- (2) A numa object is never a member variable of a non-numa object, nor a member of a numa object on a different node. Again, this placement is nonsensical as it requires the parent object to be split across different allocations which violates the standard. When creating a numa variant of C++ class C as `numa<C, k>`, the introspective template expansion expands a type T of C's field to the type `numa<T, k>`, which allows the C++ type system to enforce this invariant. Note that a `numa<C, k>` object can contain a pointer to a `numa<C', k'>` object even when $k' \neq k$. The type system allows this since the pointer itself is allocated on k, hence has the type `numa<numa<C', k'>*, k>`.

Dynamic Allocation Optimization. It is sometimes desirable to ensure the locality of an object and all its transitively reachable dependencies to ensure predictable performance. In particular, it would be ideal if we could state that, for instance, “all nodes in a `numa<Stack, 0>` are allocated on numa node 0.” However, objects may hold arbitrary references, and some references may point to non-numa pinned objects. Similarly, some methods may simply be unconvertible, for instance, a public `Stack` method with the signature `push(Node* n)` — this `Node*` parameter comes from some other allocation we do not control within the object. Enforcing referential integrity, especially on pre-existing code, is therefore not easy and requires significant manual effort.

We therefore make the pragmatic choice of enforcing a weaker constraint — a numa object, including all its fields, is allocated on the designated node — and attempt to guide the program towards ideal memory layout that satisfies referential integrity using heuristic optimizations. In particular, we observe that for a numa object, it can be assumed that internal allocations should be pinned to the same node. This optimization, implementable in the front-end, is sufficient to ensure most `Stack` nodes are colocated with its top pointer, assuming the absence of such problematic `push()` methods. Our evaluation (Sec. 6) demonstrates that our lightweight approach nonetheless confers significant performance benefits.

3.8 Transform Completion

Figure 5 shows the full transform operating on an append-only `Stack`, with both the original and transformed code. The transformed code is valid C++ with no special meaning to the word `numa`, and can be compiled using the rest of the pipeline, enabling additional numa optimizations within the backend.

4 Implementation

In this section, we provide implementation details of our clang++ based of the numa introspective type and its associated runtime.

4.1 clang++ front-end

Our implementation is written as a source-to-source transformation within clang++ version 20.0, compatible with any C++20 program. Interestingly, clang++ can only parse valid C++, which means the programmer's original code must be valid C++, and thus we need a valid definition of the numa

```

442 49 //numa definition included from our
443 50 //numa library
444 51 template<typename T, int N>
445 52 struct numa{};
446 53
447 54 struct Node{
448 55     Node* down=NULL; int val;
449 56 };
450 57
451 58 struct Stack{
452 59     Node* root=NULL;
453 60     virtual void push(int v){
454 61         Node* n = new Node();
455 62         n->val = v;
456 63         n->down = root;
457 64         root = n;
458 65     }
459 66 };
460 67
461 68 int main(){
462 69     numa<Stack,0>* s
463 70     = new numa<Stack,0>();
464 71 }
465 72 }

```

(a) Original Stack

```

73 struct Node{
74     Node* down=NULL; int val;
75 };
76 struct Stack{
77     Node* root =NULL;
78     virtual void push(int v){
79         //body of the original push
80         //method
81     }
82 }
83 struct numa<Node,0>{
84     numa<Node*,0> down=NULL;
85     numa<int,0> val;
86     // overload new/delete operators
87 };
88 struct numa<Stack,0>{
89     numa<Node*> root=NULL;
90     virtual void push(int v){
91         Node* n =
92             reinterpret_cast<Node*>(
93                 new numa<Node,0>());
94         n->val = v; n->down = root;
95         root = n;
96     }
97     // overload new/delete operators
98 };
99 int main(){
100     numa<Stack,0>* s
101     = new numa<Stack,0>();
102 }

```

(b) Stack after numa transform

Fig. 5. Complete numa transform on Stack. The empty numa definition in (a) is to allow programmers define numa types. However, it has no use once the code is transformed.

qualifier *before* actually executing our pass. Fortunately, we can use a trick: the `numa<T>` type's static behavior is equivalent to that of a child of the original `T` type. The original code can therefore be parsed using a templatized child of the parent (e.g. `template<typename T> class numa : T`), and this definition is replaced by the end of the pass with an empty (non-child) definition described in the previous section.

Once parsed, we implement our transformations as tools using the libtooling library in clang. The first tool performs the template expansion and recursive type introspection. The second tool adds the dynamic allocation optimization and appropriate reinterpret casts. Once these tools are complete, we emit fully valid, expanded C++ code, and compile using a normal compilation pipeline.

4.2 Runtime

Our numa type provides handles for using a NUMA-aware allocator. The default Linux method is the `numa_alloc_onnode()` function and is deallocated using the `numa_free()` function. Unfortunately, this allocator is known to be very slow compared to `malloc()` [4].

Instead, we utilize a modified version of the unified-memory-framework allocator [51]. This is a custom memory allocator made for heterogenous memory systems developed by Intel. The library

```

491  $T ::= \text{Int} \mid \text{Bool} \mid C \mid \text{numa}\langle T, p \rangle \quad n \in \mathbb{Z} \quad b \in \mathbb{B} \quad x, y, z \in \text{Variables} \quad f \in \text{FieldNames}$ 
492  $C \in \text{ClassNames} \quad m \in \text{MethodNames} \quad \ell \in \text{Locations} \quad p, q \in \text{Machinelds}$ 
493  $\iota = \langle \ell, p \rangle \in \text{Addresses} \quad v \in \text{Values} ::= n \mid b \mid \iota \quad e \in \text{Pure Expressions} ::= v \mid x \mid e \rightarrow f \mid \text{this}$ 
494  $E \in \text{Effectful Expressions} ::= \text{new } T(\bar{e}) \mid e \rightarrow m(\bar{e}) \mid E$ 
495  $s \in \text{Statements} ::= E \mid \text{let } x = s \text{ in } s \mid s; s \mid e \rightarrow f := E \quad M \in \text{Methods} ::= T \, m(\bar{T} \, \bar{x}) \{s\}$ 
496  $K \in \text{Constructors} ::= C(\bar{T} \, \bar{x}) \{ \text{this}.\bar{f} = \bar{x} \} \mid \text{numa}\langle C, p \rangle(\bar{T} \, \bar{x}) \{ \text{this}.\bar{f} = \bar{x} \}$ 
497  $L \in \text{Class Dfs} ::= \text{class } C [\text{extends } C] \{ \bar{T} \, \bar{f}; K; \bar{M} \}$ 
498  $\quad \mid \text{class numa}\langle C, p \rangle [\text{extends numa}\langle C, p \rangle] \{ \text{numa}\langle T, p \rangle \, \bar{f}; K; \bar{M} \}$ 
499

```

Fig. 6. FNUMA Syntax

supports combining a memory provider that does coarse grained allocations and deallocations on some designated device (e.g. on a particular NUMA node), but then provides a mechanism for more fine-grained suballocations using an allocator, in particular, `je_malloc()` [32]. In experimentation, we found that the default umf wrapper for jemalloc scaled very poorly due to disabling the per-thread caches, and we implemented a small patch to fix the issue. Despite our best efforts, our NUMA-aware umf allocator, based on jemalloc [32], is still approximately 30% slower than the default jemalloc due to a highly optimized internal “fast path” for default values. In spite of this penalty, we still found the NUMA-aware allocator to be the most performant result (see Section 6) on NUMA benchmarks, and expect additional work here could yield further improvements.

5 Formal Description

We now present a formalization of our NUMA extension to C++ using a simple object calculus inspired by Featherweight Java (FJ) formalism [29]. We call the formal language FNUMA. We prove *memory locality* theorem for FNUMA, which shows that a NUMA object is allocated entirely on a single NUMA node. FNUMA is setup to abstract over the low-level details of C++. The objective is to demonstrate the semantics of our NUMA type extension; not necessarily its C++ implementation. Consequently, the memory locality theorem follows rather straightforwardly from the construction. The language specification assumes that all template specialization has already taken place as C++’s template system is not part of the specification. All classes defined as `numa<C, k>` have already been updated through recursive typing.

5.1 Syntax

Fig. 6 shows FNUMA syntax. At the top level, FNUMA programs are a sequence of class definitions $\bar{L}$, where a class definition L defines either a regular class C or a NUMA class `numa` $\langle C, p \rangle$. As described in Sec. 3, our implementation extends C++ templates with introspection to implement the `numa` type extension. Once the templates are expanded, however, a NUMA class is treated similar to a specialized template. The process of template specialization has no bearing on operational semantics. We therefore assume that `numa` is the only template in FNUMA and all template specialization has already taken place. Like FJ [29], the class definition contains a list of typed fields $\bar{T} \, \bar{f}$, a constructor K , and a set of method definitions $\bar{M}$. A method definition M includes argument and return types and a body, which is a statement s . Since the objective of FNUMA is to demonstrate NUMA memory semantics, it extends FJ with memory and mutable fields. Consequently, FNUMA has a syntactic class of statements s , which includes (mutable) field assignments, side-effecting expressions E , and read-only expressions e . Local variables are defined using `Let` statements. Sequencing ($s; s$) is allowed.

$$\boxed{\sigma \vdash_p e \Downarrow v}$$

Store: $\sigma : \iota \mapsto \text{Value}$

$$\frac{|\bar{f}| = |\bar{v}| \quad \sigma \vdash_p e \Downarrow \iota \quad \sigma(\iota) = C(\bar{v}) \quad \text{fields}(e) = \bar{T} \bar{f}}{\sigma \vdash_p e \rightarrow f_i \Downarrow v_i} \quad [\text{ACCESS-FIELD}]$$

$$\frac{|\bar{f}| = |\bar{v}| \quad \sigma \vdash_p e \Downarrow \langle q, \ell \rangle \quad \sigma(q, \ell) = \text{numa}\langle C, q \rangle(\bar{v}) \quad \text{fields}(\text{numa}\langle C, q \rangle) = \bar{T} \bar{f}}{\sigma \vdash_p e \rightarrow f_i \Downarrow v_i} \quad [\text{ACCESS-NUMA-FIELD}]$$

Fig. 7. FNUMA operational semantics: pure expressions

A NUMA-specialized class C , written $\text{numa}\langle C, p \rangle$, is parameterized on id p of the machine/node on which the objects of that class are allocated. Its fields also have numa types with same machine id p , indicating that they are allocated on the same NUMA node as the object. Values in FNUMA include numbers n , booleans b , and addresses ι . Objects are always passed by reference (address), so a field of type $\text{numa}\langle C_1, p \rangle$ denotes the address of an object of type C_1 . The address is allocated locally on node p whereas the object maybe allocated anywhere. Basic values, e.g., integers, are expected to be boxed inside the objects of named classes when stored in numa class fields. An address ι is formally a tuple containing the NUMA node id p and the memory location l on that node. Addresses are intended to be runtime values: they are generated during evaluation when an object is allocated. Field accesses and method calls are denoted $e \rightarrow f$ and $e \rightarrow m(\bar{e})$, respectively, to emphasize that e evaluates to an address (i.e., a pointer in C++).

5.2 Operational Semantics

FNUMA operational semantics is stratified across pure expressions, effectful expressions, and statements. While the latter two could be combined into one section, we list them separately for clarity. The semantics is defined in the context of a store (σ) that maps addresses (ι) to values (v).

5.2.1 Pure Expressions. We define big-step semantics for pure expressions in Fig. 7. If some expression e evaluates to an address ι , the object at address ι is $C(\bar{v})$, class C has fields $\bar{f}$ and the length of $\bar{f}$ and $\bar{v}$ are equal then $e \rightarrow f$ evaluates to v . The same is true for when C is instead a $\text{numa}\langle C, q \rangle$ where q could be a different machine than the current machine p .

5.2.2 Effectful Expressions. We define small-step semantics for effectful expressions¹ in Fig. 8. The reduction relation $\xrightarrow{p}$ reduces the expression E under store σ to E' , returning the new store σ' on machine p . When a new object is allocated for a non-numa class, the object is allocated on the current machine p at the next available location l . When a numa object is allocated it will be allocated on the specified machine q , rather than strictly the current machine p . When executing a method call, this variables refers to the address ι of the current object.

5.2.3 Statements. Small-step semantics for statements (Fig. 9) is defined similar to effectful expressions. The let syntax allows for replacing v with the variable x . Sequencing multiple statements together is allowed, with slightly different specifications if there is a value involved as seen in Figure 9. The statements allow for setting both regular class and NUMA class fields and these may also be put into effect with expressions E and values v . The rules for statements as seen in Figure 9

¹Effectful expressions and statements include method calls that may not terminate, hence small-step semantics.

$$\boxed{\langle \sigma, E \rangle \xrightarrow{P} \langle \sigma', E' \rangle}$$

$$\frac{\text{next}(\sigma, p) = \ell \quad \iota = \langle p, \ell \rangle \quad \sigma' = \sigma[\iota \mapsto C(\bar{v})]}{\langle \sigma, \text{new } C(\bar{v}) \rangle \xrightarrow{P} \langle \sigma', \iota \rangle} \quad [\text{NEW-OBJECT}]$$

$$\frac{\text{next}(\sigma, q) = \ell \quad \iota = \langle q, \ell \rangle \quad \sigma' = \sigma[\iota \mapsto \text{numa}\langle C, q \rangle(\bar{v})]}{\langle \sigma, \text{new } \text{numa}\langle C, q \rangle C(\bar{v}) \rangle \xrightarrow{P} \langle \sigma', \iota \rangle} \quad [\text{NEW-NUMA-OBJECT}]$$

$$\frac{\sigma(\iota) = C(\bar{v}) \quad \text{mbody}(m, C) = \bar{x}.s}{\langle \sigma, \iota \rightarrow m(\bar{v}) \rangle \xrightarrow{P} \langle \sigma, [\iota/\text{this}][\bar{v}/\bar{x}]s \rangle} \quad [\text{METHOD-CALL}]$$

$$\frac{\sigma \vdash_P e \Downarrow v}{\langle \sigma, e \rangle \xrightarrow{P} \langle \sigma, v \rangle} \quad [\text{EXPRESSION}]$$

Fig. 8. FNUMA operational semantics: effectful expressions

$$\boxed{\langle \sigma, s \rangle \xrightarrow{P} \langle \sigma', s' \rangle}$$

$$\frac{}{\langle \sigma, \text{let } x = v \text{ in } s \rangle \xrightarrow{P} \langle \sigma, [v/x]s \rangle} \quad [\text{LET}]$$

$$\frac{\langle \sigma, s_1 \rangle \xrightarrow{P} \langle \sigma', s'_1 \rangle}{\langle \sigma, s_1; s_2 \rangle \xrightarrow{P} \langle \sigma', s_1; s_2 \rangle} \quad [\text{SEQUENCE}]$$

$$\frac{}{\langle \sigma, v_1; s_2 \rangle \xrightarrow{P} \langle \sigma', s_2 \rangle} \quad [\text{SEQUENCE-VALUE}]$$

$$\frac{\sigma(\iota) = C(\bar{v}) \quad \sigma' = \sigma[\iota \mapsto C(v_0 \dots v_{i-1}, v, v_{i+1}, \dots, v_{|\bar{v}|})] \quad \text{fields}(C) = \bar{T} \bar{f}}{\langle \sigma, \iota \rightarrow f_i = v \rangle \xrightarrow{P} \langle \sigma', v \rangle} \quad [\text{SET-FIELD}]$$

$$\frac{\sigma(\iota) = \text{numa}(\bar{v}) \quad \sigma' = \sigma[\iota \mapsto \text{numa}\langle C, q \rangle(v_0 \dots v_{i-1}, v, v_{i+1}, \dots, v_{|\bar{v}|})] \quad \text{fields}(\text{numa}\langle C, q \rangle) = \bar{T} \bar{f}}{\langle \sigma, \iota \rightarrow f_i = v \rangle \xrightarrow{P} \langle \sigma', v \rangle} \quad [\text{SET-NUMA-FIELD}]$$

$$\frac{\sigma \vdash_P e \Downarrow \iota}{\langle \sigma, e \rightarrow f := E \rangle \xrightarrow{P} \langle \sigma', \iota \rightarrow f := E \rangle} \quad [\text{SET-FIELD-LHS-EXPR}]$$

$$\frac{\langle \sigma, E \rangle \xrightarrow{P} \langle \sigma', E' \rangle}{\langle \sigma, \iota \rightarrow f := E \rangle \xrightarrow{P} \langle \sigma', \iota \rightarrow f := E' \rangle} \quad [\text{SET-FIELD-RHS-EXPR}]$$

Fig. 9. FNUMA operational semantics: statements

shows that these operations work the same way with regular class instances as they do with numa class instances.

5.3 Safety

In Sec. 3.7, we describe two invariants our template-expanded C++ should satisfy. The first invariant, which prohibits the stack allocation of NUMA objects, vacuously holds in FNUMA since our operational semantics never allocates objects on stack. The second invariant, which requires a $\text{numa}\langle C, k \rangle$ object to always be allocated on node k , can be formalized as the following well-formedness condition on the store σ :

$$\text{well-formed}(\sigma) \stackrel{\text{def}}{=} \forall p, q, l, C, \bar{v}. \sigma(p, l) = \text{numa}\langle C, q \rangle(\bar{v}) \Rightarrow p = q$$

Observe that the evaluation of the expression $e \rightarrow f$ in the rule ACCESS-NUMA-FILED gets stuck if the σ is not well-formed. We show that this cannot happen unless the evaluation starts with an ill-formed store.

THEOREM 5.1 (MEMORY LOCALITY).

$$\forall p, \sigma, s, \sigma', s'. \text{well-formed}(\sigma) \wedge \langle \sigma, s \rangle \xrightarrow{p} \langle \sigma', s' \rangle \Rightarrow \text{well-formed}(\sigma')$$

PROOF. Proven in Coq by induction on $\langle \sigma, s \rangle \xrightarrow{p} \langle \sigma', s' \rangle$. Requires the corresponding lemma on the evaluation of effectful expressions E . $\square$

Considering an empty store is trivially well-formed, no evaluation starting with an empty store results in an ill-formed store, hence memory locality of numa objects is always maintained. Note that we do not extend C++'s type system in any way. Equipping FNUMA with a C++-like type system (similar to [29]) will also let us prove the absence of stuck states.

6 Evaluation

In this section, we present the evaluation of our type system using two benchmarks: a transactional binary search benchmark and another a YCSB benchmark we adapted to be NUMA-aware and run on a multi-socket machine. Specifically, we evaluate our type system and answer two research questions:

- RQ1: Does our introspective type expansion reduce the manual annotation required to take advantage of fine-grained NUMA thread placement?
- RQ2: Does the additional control over NUMA allocation translate to performance benefits across diverse workloads and machine configurations?

Th. Config	DS. Config	Description
<i>numa th.</i>	<i>numa ds.</i>	A thread pinned to NUMA node k operating on data allocated on the same node k , enabling local memory access.
<i>numa th.</i>	<i>regular ds.</i>	A thread pinned to NUMA node k accessing data allocated without NUMA awareness.
<i>regular th.</i>	<i>numa ds.</i>	A standard thread (not pinned) accessing data allocated on a specific NUMA node k .
<i>regular th.</i>	<i>regular ds.</i>	A standard thread accessing a data allocated with standard <code>malloc</code> , with no NUMA affinity on either side.

Table 1. Thread and Data Structure Configuration Combinations

6.1 Experimental Setup

We conduct our experiments on two machines of varying NUMA configurations. The first machine (**small**) is a dual-socket NUMA system equipped with two Intel Xeon Silver processors, each

featuring 20 physical cores (40 threads with Hyper-Threading enabled), totaling 80 logical cores. The machine consists of 128 GB of RAM, with 64 GB allocated to each NUMA node. The idle latency difference between local and remote accesses on the two NUMA nodes is about 1.5x.

To test the scalability of our approach, we also replicated our experiments on a much larger dual socket AMD EPYC 7763 64-Core NUMA machine with 128 physical cores (256 threads with Hyper-Threading enabled) split across 8 NUMA nodes (**large**). Each node has a 64 GB main memory attached to it bringing the total available memory to 512 GB. The maximum latency difference between local and remote memory across the 8 nodes is about 3x.

Both machines run a Linux kernel but our 8 NUMA node machine runs a specialized Linux kernel tailored for high performance computing called SUSE Linux Enterprise Server (SLES) [52].

We evaluate the lines of code expansion and performance benefits of our data type extension and introspective typing compiler optimization on an open-source C++20 simple, sequential data structure suite [43], and a NUMA-aware version of the Yahoo! Cloud Serving Benchmark (YCSB) [16].

In both benchmarks, we modified the source code and added the `numa` and `thread_numa<k>` qualifier to the included unit tests to trigger our introspective expansion and validate run-time performance behavior. As part of our experimental validation, we had to fix a few small bugs in the code, namely filling the `remove()` stub in the `BinaryTree` implementation in the simple data structures benchmark. The benchmark itself was otherwise unchanged.

Class	Original LOC	Expanded LOC
Stack	73	212
Queue	71	208
BinarySearchTree	219	639
LinkedList	199	620
Node	48	206
BinaryNode	36	200

Table 2. Data structure suite source LOC before and after source-to-source expansion.

Class	Original LOC	Expanded LOC
HashTable	165	567
HashNode	20	138

Table 3. NUMA-aware YCSB source LOC before and after source-to-source expansion. The only two files expanded are the headers that define the data structures for the workloads.

6.2 Code Expansion

Our automated NUMA-aware expansion increases the overall lines of code of both the data structures suite and NUMA-aware YCSB benchmark by 1.2-1.3x when compiling for two numa nodes. Our compile time specialization of numa types directly injects all explicitly specialized classes in the header files where the respective class for the qualified type is defined. Table 2 and Table 3 show the number of lines before and after our introspective transformation for each specialized header file, for the simple data structures benchmark and the NUMA-aware YCSB benchmark, respectively. This investigation

demonstrates that our introspective type system does useful and significant expansions, that otherwise would have been done manually by the programmer, to achieve locality and NUMA-awareness.

6.3 Throughput Performance

As a demonstration of our numa type's performance, we use the the binary search tree program from the data structures suite and different configurations of all YCSB workloads to build a multi-threaded transactional processing engine.

The transactional processing engine is designed to evaluate the performance implications of NUMA-aware programming of our benchmarks under different thread and data placement strategies. Our thread configuration can be *regular*, where threads are created using `std::thread` with no specific NUMA affinity, or it can be *numa*, where threads use our extended C++ type `thread_numa<k>`,

to describe a specific NUMA node to run on that can co-locate computation and data. The same approach is used for the data structure configuration, where we can have a *regular* data structure implying a standard NUMA agnostic data placement on the heap, or a *numa* data structure where we use our `numa<T, k>` type to specify that the data structure, `T` is allocated on NUMA node `k`. Different combinations of these thread and data structure configurations and what they mean for the locality of memory accesses are described in Table 1.

6.3.1 Transactional Benchmark. Our transactional benchmark explores NUMA-aware performance for small transactions among large collections of key-value stores in an imitation of high-throughput account transfers and balance queries. We use large arrays of BinarySearchTrees that store keys and values, with a lock protecting each index. Lookups are executed using a two step hash, first into the index, then into the tree. Transactions are executed using two-phase locking, and transfer values between keys.

All threads perform queries on only half of the store, representing a machine partition amenable to a two-sided NUMA architecture and easily implementable via work queues that dispatch client lookup requests. However, transactions on the store may need to perform cross-node transactions (e.g. a transfer from a locally stored index to a remote), resulting in a local thread modifying part of the store that primarily resides on the remote node.

All threads that can be spawned in the system perform 80% (local) account query operations and 20% (mixed) transaction operations between arbitrary accounts. When applicable for the experimental configuration, the threads and data structures are appropriately co-located, and we create a large enough working set to ensure that it cannot simply be cached by the machine.

For our experiment on the small dual-socket, two-node machine, we initialize 1 million indices each with 80 keys, for a total memory footprint of around 2 GB. We measure total operations performed in 10 minutes.

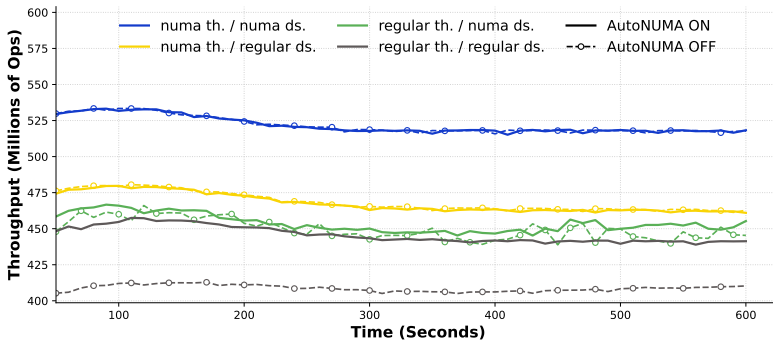


Fig. 10. Transactional benchmark throughput performance over time with and without AutoNUMA on **small** machine.

Figure 10 shows the throughput achieved by the four configurations of thread and data structure when AutoNUMA [1] is on and off. As expected, the best performing configuration is the *numa th. / numa ds.* configuration which has 27% performance improvement as compared to our NUMA-unaware baseline *regular th. / regular ds.* configuration when AutoNUMA is disabled and up to 17% improvement when AutoNUMA is enabled. This improvement comes from using our NUMA-aware types where all data structures and their associated memory allocations at runtime are pinned to specific numa nodes together with the threads that execute the random read and transaction operations on them.

The *numa th. / regular ds.* approach ranks second, as the default first touch allocation policy of the operating system will localize the data with the pinned threads at initialization. However, as the run progresses, some corruption of data due to non-local transactions is bound to occur since the data itself is not pinned. The *numa th. / numa ds.* configuration, which is the correct way of using our numa data type in programming, still shows a 15% performance improvement over the *numa th. / regular ds.* configuration. The *regular th. / numa th.* configuration is subject to the load balancing mechanism in Linux. When AutoNUMA is disabled, since the threads are not pinned, we observe a fluctuation in throughput values as each thread has a 50% chance of getting scheduled to a far away node after a context switch. AutoNUMA will try to correct this fluctuation when it is enabled as it tries to schedule threads closer the data they are operating on.

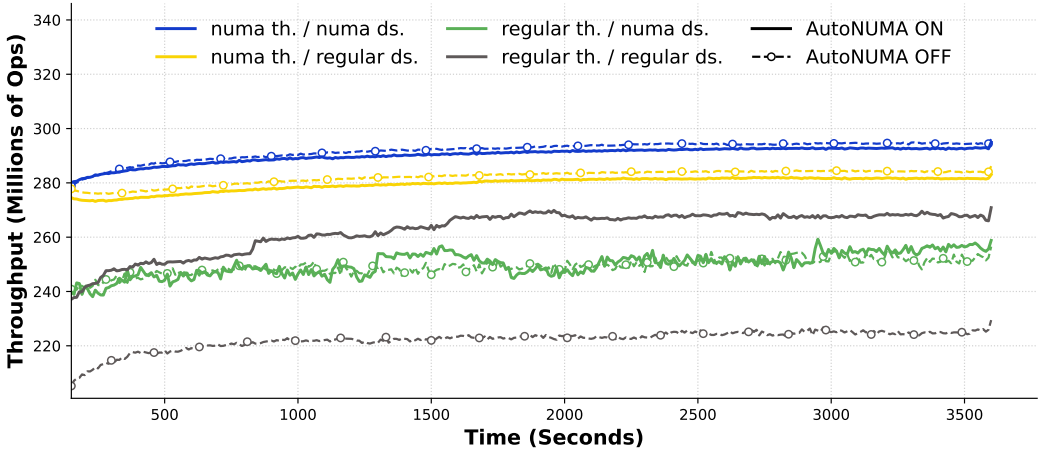


Fig. 11. Transactional benchmark throughput performance over time with and without AutoNUMA on **large** machine.

We increased our memory footprint by 3x and ran it on our large dual-socket(8 NUMA node) AMD EPYC processor to see how our benchmark scales. Figure 11 shows the throughput performance of the experiments when run for 1 hour. The *numa th. / numa ds.* configuration, when AutoNUMA is disabled, shows up to 31% improvement over the *regular th. / regular ds.* configuration. Turning AutoNUMA on, especially for a longer run, can have a degrading effect in performance. This is because of the side effect of the AutoNUMA profiler moving unrelated, unpinned data pages that are repeatedly accessed, such as locks and other globals.

6.3.2 NUMA-aware YCSB. The Yahoo! Cloud Serving Benchmark (YCSB) [16] is a widely used benchmark for evaluating the performance of different cloud databases such as Cassandra [36], HBase [7] and more under heavy, real-world traffic. It defines a set of workloads that simulate different types of operations that characterize memory accesses for datacenters. YCSB uses a zipfian distribution for its access pattern — hot keys are accessed far more than cold keys. While YCSB is commonly used for benchmarking, the workload is uniform and undifferentiated for any NUMA-style partitioning.

To isolate the effect of NUMA memory latency, we adapt YCSB for our NUMA experiments. We pin half the threads on a given NUMA node and have them do a 50/50 mix of local and remote operations, while the remaining half performs a 100% local operations. This configuration emulates running several YCSB configurations in parallel, some of which exhibit locality. We also introduced

a new workload, workload AD, to test how our system performs in a mix of different workloads where half the threads on one numa node runs workload A with a 50/50 split of local and remote operations and the other half runs workload D with 100% local operations. Figure 12 illustrates the distribution of local and remote accesses for workload AD but the same distribution applies for the homogenous workloads as well. Table 4 describes the read-write patterns of the YCSB workloads.

Workload	Operations	Application
A	50% read 50% update	Application that tracks user activities per session, like updating timestamps associated with user actions.
B	95% read 5% update	Record tagging over a database. Most operations are reading tags with occasional updates.
C	100% read	Caching user profiles, where most operations are built and modified in another system.
D	95% read 5% insert	Posting a new status in social media where most people are reading latest status updates.
E	95% scan 5% insert	Comment sections and threaded conversations in social media where the system performs scan to retrieve related data at once.
F	50% read 50%	Tracking user transactions in a financial application that requires knowing the previous state of the data.
AD	72.5% read 25% update 2.5% insert	A synthetic workload of combining a read heavy workload with a write heavy workload.

Table 4. YCSB workload read-write distribution. Workload AD is a mix of workload A and workload D.

Our experiment on the small 2-node Intel machine spawns 80 threads (the maximum limit) and creates 1000 hash tables with 133300 buckets each, where each bucket holds a linked list of up to 100 million, 32 Byte entries that point to approximately 10 Byte strings. This brings the total memory footprint of workset to around 10 GB which exceeds the capacity of the last level cache. We run each workload for 20 minutes and measure the total operations performed. Figure 13 shows the normalized throughput achieved by all workloads under the four thread and data structure configurations with AutoNUMA on and off.

Our system achieves its peak throughput performance (of up to 21%) under workload C. Because workload C consists entirely of read operations, the data structure is mostly stable in memory and is not exposed to cross node cache invalidation and other inherent effects that a write operation can introduce. This absence of noise brings out the overhead of the hardware interconnect latency on the critical-path of our throughput measurement. This overhead is also what our proposed type system extension intends to alleviate. It is also worth noting that the *numa th. / regular ds.* configuration achieves a

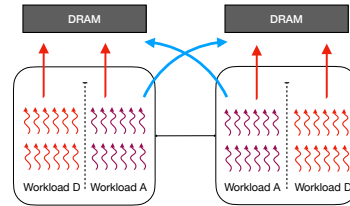


Fig. 12. NUMA-aware YCSB workload locality distribution.

comparable effect in this scenario. This occurs because the hash table workset is initialized by `thread_numa` threads, which takes advantage of the operating system's default first-touch policy to create memory locality at startup. Since workload C never allocates new nodes or update the keys at runtime, this initialization layout never degrades.

In workloads that involve writes the system our *numa th. / numa ds.* configurations still show 7-10% performance improvements compared to *regular th. / regular ds.* configurations. In this scenario, we also start seeing the benefits of using *numa th. / numa ds.* over *numa th. / regular ds.* as not pinning the data will introduce more cross node pollution because the default allocator is not capable of binding data to the right nodes.

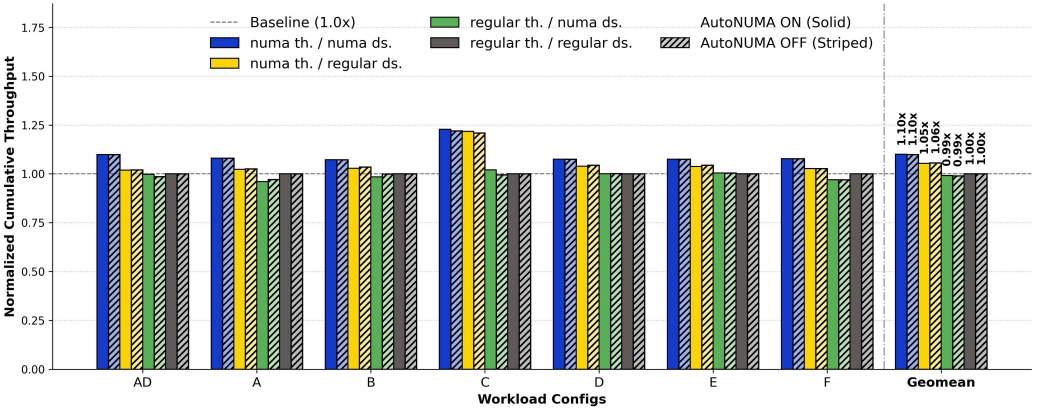


Fig. 13. NUMA-aware YCSB throughput comparison of the four configurations of thread and data structure with AutoNUMA on and off on the **small** machine. All throughput values are normalized to the *regular th. / regular ds.* configuration as a baseline.

In *regular th. / numa ds.* configurations, there is no benefit of locality coming from the first touch operating system policy. Additionally, the slower numa allocator will tax the throughput performance. When AutoNUMA is turned on, it introduces even more uncertainty to the system as the AutoNUMA telemetry can conflict with the load balancing scheduler which is now free to take a thread from one node and assign it to another. For this reason, we strongly discourage numa types to be used in a manner where data is pinned to numa nodes but the threads are not.

The memory access behavior of the four configuration of workload AD is shown in Figure 14 and 15. The number of local accesses for the *numa th. / numa ds.* configuration is about 3.5x that of remote accesses regardless of enabling or disabling AutoNUMA. Most of the other configurations show little to no difference in the number of local and remote accesses. However, from Figure 15 (c), it can be seen that AutoNUMA will try to alleviate the overlapping of local and remote accesses in the case of *regular th. / numa ds.* as it tries to place data where the threads are scheduled.

Figure 16 shows the results of running the same experiment for two hours on the large dual-socket 8 numa node, AMD EPYC machine. This machine has a longer latency across its two physical socket and so our numa types, when used correctly, can achieve a higher throughput across all workloads. However, it should be noted that in this scenario, where we are running the benchmark for a longer period of time, AutoNUMA can have an adverse effect. Locks and other structures that maybe used repeatedly but are not of numa types can end up polluting cross node pages. Enabling AutoNUMA can further interfere with performance, especially when being used in the *numa th. / numa ds.* configuration, as checking which pages and threads can be moved becomes meaningless,

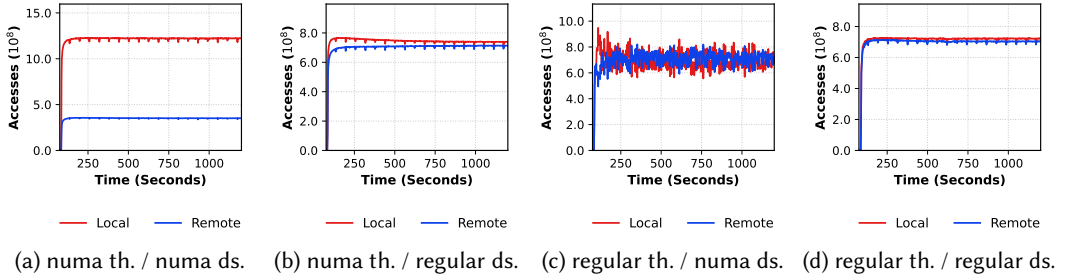


Fig. 14. Remote and local DRAM accesses over time for the four thread/data structure configurations when AutoNUMA is off (small machine).

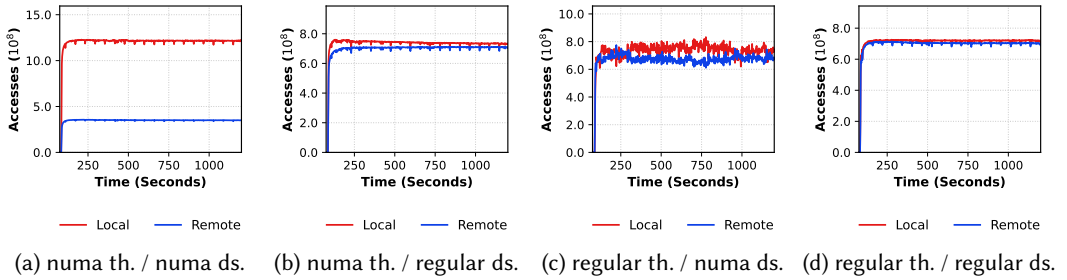


Fig. 15. Remote and local DRAM accesses over time for the four thread/data structure configurations when AutoNUMA is on (small machine).

if they are both pinned. For *numa th. / regular ds.* configuration, AutoNUMA further worsens the throughput as the remote operations will hint AutoNUMA to migrate pages back and forth between two nodes. This ping-pong effect hinders AutoNUMA-on performance for read heavy workloads. In general, we advice against having AutoNUMA enabled when using our extended type system.

In summary, Figures 10, 11, 13, 14, 15 and 16 demonstrates the performance advantages of our extended type system across a diverse set of workloads and under the presense of AutoNUMA interventions.

7 Related Work

There are many works on NUMA optimization that propose NUMA-aware programming models or data structures, or operation system policies. They all differ from our work as they use vastly different methods, provide the programmer with little to no topology control, or lack generalization.

7.1 NUMA Type systems and Languages

One common method of NUMA optimization is using NUMA-aware type systems or programming languages. These systems are generally full languages, or, at the very least, intend developers to rebuild systems from scratch — unlike our system which can transform existing code into a NUMA-aware application.

Among those, the closest to our work is Franco and Drossopoulou's, which proposes and formalizes a new type system (and language) that grants the programmer control over the topology of the system as well as the permissible interactions between objects via a combination of ownership

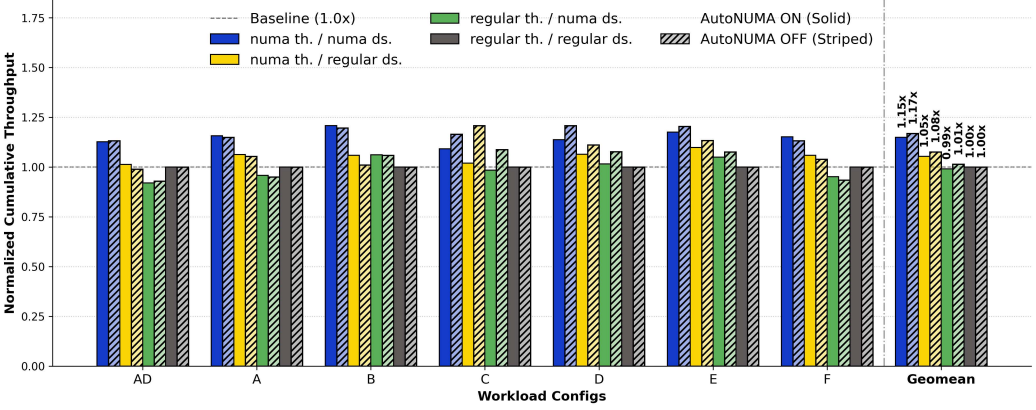


Fig. 16. Perlmutter Throughput comparison of the four configurations of thread and data structure with AutoNUMA on and off on a the **large** machine. All throughput values are normalized to the *regular th. / regular ds.* configuration as a baseline.

types and behavioral types [24]. Using this type system, the programmer can reduce remote accesses by placing related data in close local proximity and limiting allowed remote interactions. In contrast, we build on C++ instead of creating a new type system. We also only use hierarchical ownership types, without the need for behavioral types, to introspectively pin all sibling objects to the same NUMA node as the parent object, thus automatically pinning a cluster of related data to the same node.

Our work is also comparable to X10’s place types [15, 48], which implement a dependent type system which embed locality information within types. Like our work, this project attaches locality information in the type system, but with dramatically different goals — the system exists to prohibit cross-locality accesses, as opposed to discovering and optimizing them. As such, place types result in a more restrictive type system, as opposed to our numa type, which is as permissive as the underlying C++ host language.

Another common method for NUMA-awareness is by using libraries, APIs, or frameworks built around task scheduling [42, 44, 46, 47]. However, they rely on a task-scheduling paradigm, as opposed to our general purpose interface, using the clear data dependencies of the task-scheduling model to better organize computation and memory, either by attaching tasks to particular memory regions, such as with MxTasks [42], Chapel [14], or Legion [5], or extracting the data dependencies between tasks, as seen in Jade [46, 47].

7.2 Other NUMA Optimization Methods

There are other methods that do not rely heavily on code-level optimizations, but they do not generalize well, nor do they provide the programmer control over the program topology. These include the use of OS policies or NUMA-aware data structures.

7.2.1 OS and OS Policies. In general, OS policies in NUMA optimization is to develop memory management algorithms that increase the locality of relevant data, as Bolosky et al. suggests [8]. This can be achieved by moving data as AutoNUMA [1], which, as described in Section 2, migrates both data and processes according to the program’s memory access statistics, an approach that can be refined with various profiling techniques [35, 53] or applied to CXL-based machines [38].

Another common way to increase locality is by duplicating data [8, 12, 19, 21]. These works differ in the way they approach replication, for instance by replicating read-only pages [8, 12], or implementing a page-based coherence [19, 21]. Some works go beyond policies, and propose new OS architectures geared towards NUMA systems [6, 9, 26, 34].

7.2.2 Data Structures. When looking at the literature there have been many forays into NUMA optimization through the use of data structures. These contributions can be broadly separated into novel data structures that take advantage of the architecture or novel techniques that allow such data structures to exist. Software and hardware transactional memory have both been able to allow for easier use of NUMA but do not reach the level of optimization a proper data structure could [11, 28, 50]. Other techniques like read-copy update and its extension read-log-update [40, 41] provide another alternative to concurrency primitives for building a specialized data structure. Other approaches, such as Asynchronized Concurrency [22] or message passing paradigms [49], involves building concurrent data structures using certain programming patterns in which the resulting work will inherently be optimized for NUMA architectures. Another path that has been explored previously is synthesizing [13, 27, 54] improvements to current code like the addition of speculation or being able to generate correct concurrent code out of code written with certain specifications so that performance improvements can be achieved without a specialized data structure, in both scenarios.

8 Conclusion and Future Work

In this work, we presented an introspective typing system for C++ that provides fine-grained control over memory placement and thread affinity in NUMA architectures. By extending the C++ type system, with our numa types, we allow programmers to explicitly encode memory locality and thread placement semantics directly in the language. Our clang++ based compiler front end then performs a source-to-source transformation that recursively specializes user defined types to ensure consistent allocation of all objects across memory hierarchies.

We formally describe our type extension and prove memory locality guarantees in Coq. Our implementation demonstrates tangible performance benefits in a transactional binary search tree benchmark, achieving up to 31% performance improvement by aligning computation and memory access locality with minimal programmer overhead. The same solution, when adapted to a NUMA-aware YCSB achieves up to 20% performance improvement.

In future work, we aim to broaden our approach beyond NUMA by exploring other types of memory technologies. This includes persistent memory for fault-tolerant systems and secure memory for enclave-based computation. Programming with both these memories comes with its own requirements where programmers must define persistency behavior for their programs in persistent memory [45] and trusted compute base reduction in secure memory [39]. We are also interested in expanding our type system and introspective typing with richer qualifier inference capabilities and automatic transformations of code to some these memory types.

9 Data Availability Statement

A complete artifact containing the source code of our NUMA-aware type extension exists, but is currently being withheld from this submission to maintain anonymity. The artifact includes our clang++ code transformation tools and various evaluation benchmarks for the system. There are no specific limitations to the artifact. We intend to submit it in full for the Artifact Evaluation process. Upon acceptance, the final version of the paper will include a direct link to the non-anonymized source code repository.

References

- [1] Autonuma. <https://lwn.net/Articles/488709/>. Accessed: 2026-03-14.
- [2] The linux kernel archives. <https://www.kernel.org/doc/html/v4.18/vm/numa.html>. Accessed: 2026-03-14.
- [3] numactl. <https://linux.die.net/man/8/numactl>. Accessed: 2025-03-12.
- [4] numactl. <https://man7.org/linux/man-pages/man3/numa.3.html>. Accessed: 2025-03-12.
- [5] Michael Bauer, Sean Treichler, Elliott Slaughter, and Alex Aiken. Legion: Expressing locality and independence with logical regions. In *SC '12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, pages 1–11, 2012.
- [6] Andrew Baumann, Paul Barham, Pierre-Evariste Dagand, Tim Harris, Rebecca Isaacs, Simon Peter, Timothy Roscoe, Adrian Schüpbach, and Akhilesh Singhanian. The multikernel: a new os architecture for scalable multicore systems. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles, SOSP '09*, page 29–44, New York, NY, USA, 2009. Association for Computing Machinery.
- [7] Vijayalakshmi Bhupathiraju and Ravi Prasad Ravuri. The dawn of big data - hbase. In *2014 Conference on IT in Business, Industry and Government (CSIBIG)*, pages 1–4, 2014.
- [8] W. Bolosky, R. Fitzgerald, and M. Scott. Simple but effective techniques for numa memory management. *SIGOPS Oper. Syst. Rev.*, 23(5):19–31, November 1989.
- [9] Silas Boyd-Wickizer, Haibo Chen, Rong Chen, Yandong Mao, Frans Kaashoek, Robert Morris, Aleksey Pesterev, Lex Stein, Ming Wu, Yuehua Dai, Yang Zhang, and Zheng Zhang. Core: An operating system for many cores. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation, OSDI'08*, pages 43–57, Berkeley, CA, USA, 2008. USENIX Association.
- [10] Timothy Brecht. On the importance of parallel application placement in numa multiprocessors. In *Symposium on Experiences with Distributed and Multiprocessor Systems (SEDMS IV)*, pages 1–18, 1993.
- [11] Trevor Brown, Alex Kogan, Yossi Lev, and Victor Luchangco. Investigating the performance of hardware transactions on a multi-socket machine. In *Proceedings of the 28th ACM Symposium on Parallelism in Algorithms and Architectures, SPAA '16*, page 121–132, New York, NY, USA, 2016. Association for Computing Machinery.
- [12] Edouard Bugnion, Scott Devine, Kinshuk Govil, and Mendel Rosenblum. Disco: running commodity operating systems on scalable multiprocessors. *ACM Trans. Comput. Syst.*, 15(4):412–447, November 1997.
- [13] Irina Calciu, Siddhartha Sen, Mahesh Balakrishnan, and Marcos K. Aguilera. Black-box concurrent data structures for numa architectures. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '17*, page 207–221, New York, NY, USA, 2017. Association for Computing Machinery.
- [14] B.L. Chamberlain, D. Callahan, and H.P. Zima. Parallel programmability and the chapel language. *The International Journal of High Performance Computing Applications*, 21(3):291–312, 2007.
- [15] Satish Chandra, Vijay Saraswat, Vivek Sarkar, and Rastislav Bodik. Type inference for locality analysis of distributed data structures. In *Proceedings of the 13th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPoPP '08*, page 11–22, New York, NY, USA, 2008. Association for Computing Machinery.
- [16] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing, SoCC '10*, pages 143–154, New York, NY, USA, 2010. ACM.
- [17] Julita Corbalan, Xavier Martorell, and Jesus Labarta. Evaluation of the memory page migration influence in the system performance: the case of the sgi o2000. In *Proceedings of the 17th annual international conference on Supercomputing*, pages 121–129, 2003.
- [18] Jonathan Corbet. Driver porting: Mutual exclusion with seqlocks. lwn.net/Articles/22818/, feb 2003.
- [19] A. Cox and R. Fowler. The implementation of a coherent memory abstraction on a numa multiprocessor: experiences with platinum. In *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles, SOSP '89*, page 32–44, New York, NY, USA, 1989. Association for Computing Machinery.
- [20] cpp libraries. Boost.hana. https://www.boost.org/doc/libs/1_77_0/libs/hana/doc/html/index.html, 2016. Accessed: 2025-03-22.
- [21] Mohammad Dashti, Alexandra Fedorova, Justin Funston, Fabien Gaud, Renaud Lachaize, Baptiste Lepers, Vivien Quema, and Mark Roth. Traffic management: a holistic approach to memory placement on numa systems. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '13*, page 381–394, New York, NY, USA, 2013. Association for Computing Machinery.
- [22] Tudor David, Rachid Guerraoui, and Vasileios Trigonakis. Asynchronized concurrency: The secret to scaling concurrent search data structures. In *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '15*, page 631–644, New York, NY, USA, 2015. Association for Computing Machinery.

- [23] Andi Drebes, Antoniu Pop, Karine Heydemann, Nathalie Drach, and Albert Cohen. Numa-aware scheduling and memory allocation for data-flow task-parallel applications. *SIGPLAN Not.*, 51(8), February 2016.
- [24] Juliana Franco and Sophia Drossopoulou. Behavioural types for non-uniform memory accesses. *arXiv preprint arXiv:1602.03599*, 2016.
- [25] Yehonatan Fridman, Suprasad Mutalik Desai, Navneet Singh, Thomas Willhalm, and Gal Oren. Cxl memory as persistent memory for disaggregated hpc: A practical approach. In *Proceedings of the SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, SC-W '23, page 983–994, New York, NY, USA, 2023. Association for Computing Machinery.
- [26] Benjamin Gamsa, Orran Krieger, Jonathan Appavoo, and Michael Stumm. Tornado: Maximizing locality and concurrency in a shared memory multiprocessor operating system. 1999.
- [27] Peter Hawkins, Alex Aiken, Kathleen Fisher, Martin Rinard, and Mooly Sagiv. Concurrent data representation synthesis. *SIGPLAN Not.*, 47(6):417–428, June 2012.
- [28] Maurice Herlihy and J. Eliot B. Moss. Transactional memory: Architectural support for lock-free data structures. In *Proceedings of the 20th Annual International Symposium on Computer Architecture*, ISCA '93, pages 289–300, New York, NY, USA, 1993. ACM.
- [29] Atshushi Igarashi, Benjamin Pierce, and Philip Wadler. Featherweight java: a minimal core calculus for java and gj. In *Proceedings of the 14th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA '99, page 132–146, New York, NY, USA, 1999. Association for Computing Machinery.
- [30] Joseph Izraelevitz, Jian Yang, Lu Zhang, Juno Kim, Xiao Liu, Amirsaman Memaripour, Yun Joon Soh, Zixuan Wang, Yi Xu, Subramanya R Dullloor, et al. Basic performance measurements of the intel optane dc persistent memory module. *arXiv preprint arXiv:1903.05714*, 2019.
- [31] Joseph Izraelevitz, Jian Yang, Lu Zhang, Amirsaman Memaripour, Yun Joon Soh, Subramanya R. Dullloor, Jishen Zhao, Juno Kim, Xiao Liu, Zixuan Wang, Yi Xu, and Steven Swanson. Basic performance measurements of the intel optane dc persistent memory module. *arXiv preprint arXiv:1903.05714*, 2019.
- [32] jemalloc. jemalloc. Available at <http://jemalloc.net>, 2025. Accessed: February 25, 2025.
- [33] Tobias Schwinger Joel de Guzman, Dan Marsden. Boost.fusion. https://www.boost.org/doc/libs/1_85_0/libs/fusion/doc/html/, 2012. Accessed: 2025-03-22.
- [34] Orran Krieger, Marc Auslander, Bryan Rosenburg, Robert W. Wisniewski, Jimi Xenidis, Dilma Da Silva, Michal Ostrowski, Jonathan Appavoo, Maria Butrico, Mark Mergen, Amos Waterland, and Volkmar Uhlig. K42: building a complete operating system. *SIGOPS Oper. Syst. Rev.*, 40(4):133–145, April 2006.
- [35] Renaud Lachaize, Baptiste Lepers, and Vivien Quéma. Memprof: a memory profiler for numa multicore systems. In *Proceedings of the 2012 USENIX Conference on Annual Technical Conference*, USENIX ATC'12, page 5, USA, 2012. USENIX Association.
- [36] Avinash Lakshman and Prashant Malik. Cassandra: a decentralized structured storage system. *SIGOPS Oper. Syst. Rev.*, 44(2):35–40, April 2010.
- [37] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. Pond: Cxl-based memory pooling systems for cloud platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS 2023, page 574–587, New York, NY, USA, 2023. Association for Computing Machinery.
- [38] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. Pond: Cxl-based memory pooling systems for cloud platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS 2023, page 574–587, New York, NY, USA, 2023. Association for Computing Machinery.
- [39] Joshua Lind, Christian Priebe, Divya Muthukumaran, Dan O'Keefe, Pierre-Louis Aublin, Florian Kelbert, Tobias Reiher, David Goltzsche, David Eysers, Rüdiger Kapitza, Christof Fetzer, and Peter Pietzuch. Glamdring: Automatic application partitioning for intel SGX. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*, pages 285–298, Santa Clara, CA, July 2017. USENIX Association.
- [40] Alexander Matveev, Nir Shavit, Pascal Felber, and Patrick Marlier. *Read-log-update: a lightweight synchronization mechanism for concurrent programming*, page 168–183. Association for Computing Machinery, New York, NY, USA, 2015.
- [41] P. E. Mckenney and J. D. Slingwine. Read-copy update: using execution history to solve concurrency problems. In *Parallel and Distributed Computing and Systems*, pages 509–518, October 1998.
- [42] Jan Mühlig and Jens Teubner. Mxtasks: How to make efficient synchronization and prefetching easy. In *Proceedings of the 2021 International Conference on Management of Data*, SIGMOD '21, page 1331–1344, New York, NY, USA, 2021. Association for Computing Machinery.

- [43] nmante. Data-structures. <https://github.com/nmante/Data-Structures>, 2017. Accessed: 2025-03-22.
- [44] OpenMP Architecture Review Board. The openmp api. Available at <https://www.openmp.org>, 2024. Accessed: February 25, 2025.
- [45] Steven Pelley, Peter M Chen, and Thomas F Wenisch. Memory persistency. *ACM SIGARCH Computer Architecture News*, 42(3):265–276, 2014.
- [46] Martin C. Rinard. Locality optimizations for parallel computing using data access information. *International Journal of High Speed Computing*, 09(02):161–179, 1997.
- [47] M.C. Rinard, D.J. Scales, and M.S. Lam. Jade: a high-level, machine-independent language for parallel programming. *Computer*, 26(6):28–38, 1993.
- [48] Vijay A. Saraswat, Vivek Sarkar, and Christoph von Praun. X10: concurrent programming for modern architectures. In *Proceedings of the 12th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPOPP ’07, page 271, New York, NY, USA, 2007. Association for Computing Machinery.
- [49] Omid Shahmirzadi, Thomas Ropars, and André Schiper. High-throughput maps on message-passing manycore architectures: Partitioning versus replication. In *European Conference on Parallel Processing*, 2014.
- [50] Nir Shavit and Dan Touitou. Software transactional memory. In *Proceedings of the 1995 ACM Symposium on Principles of Distributed Computing*, PODC ’95, pages 204–213, New York, NY, USA, 1995. ACM.
- [51] Open Source. Intel unified memory allocator. <https://github.com/oneapi-src/unified-memory-framework>, 2025. Accessed: 2025-03-22.
- [52] SUSE. SUSE Linux Enterprise Server. <http://suse.com/>. Accessed: 2026-03-14.
- [53] Ben Verghese, Scott Devine, Anoop Gupta, and Mendel Rosenblum. Operating system support for improving data locality on cc-numa compute servers. *SIGPLAN Not.*, 31(9):279–289, September 1996.
- [54] Lingxiang Xiang and Michael Lee Scott. Compiler aided manual speculation for high performance concurrent data structures. *SIGPLAN Not.*, 48(8):47–56, February 2013.